Table of Contents

Harness engineering for coding agent users

To let coding agents work with less supervision, we need ways to increase our confidence in their result. As software engineers, we have a natural trust barrier with AI-generated code - LLMs are non-deterministic, they don't know our context, and they don't really understand the code, they think in tokens. This article explores a mental model that brings together emerging concepts from context and harness engineering to build that trust.

02 April 2026

Birgitta Böckeler

Birgitta is a Distinguished Engineer and AI-assisted delivery expert at Thoughtworks. She has over 20 years of experience as a software developer, architect and technical leader.

generative AI

Contents

- Feedforward and Feedback
- Computational vs Inferential
- The steering loop
- Timing: Keep quality left
- Regulation categories
  - Maintainability harness
  - Architecture fitness harness
  - Behaviour harness
- Harnessability
- Harness templates
- The role of the human
- A starting point - and open questions

Sidebars

- Metaphors only go so far
- How does harness engineering relate to context engineering?
- Ambient affordances
- Ashby's Law

This article updates an earlier memo outlining my first impressions of harness engineering.

The term harness has emerged as a shorthand to mean everything in an AI agent except the model itself - Agent = Model + Harness. That is a very wide definition, and therefore worth narrowing down for common categories of agents. I want to take the liberty here of defining its meaning in the bounded context of using a coding agent. In coding agents, part of the harness is already built in (e.g. via the system prompt, or the chosen code retrieval mechanism, or even a sophisticated orchestration system). But coding agents also provide

us, their users, with many features to build an outer harness specifically for our use case and system.

Metaphors only go so far

It has been pointed out to me that wrapping harnesses around harnesses doesn't make sense: “Have you ever tried to put a harness on the inside of a dog?” So this somewhat stretches the metaphor, but I'm happy to accept that if it proves useful to navigate the usage of this word.

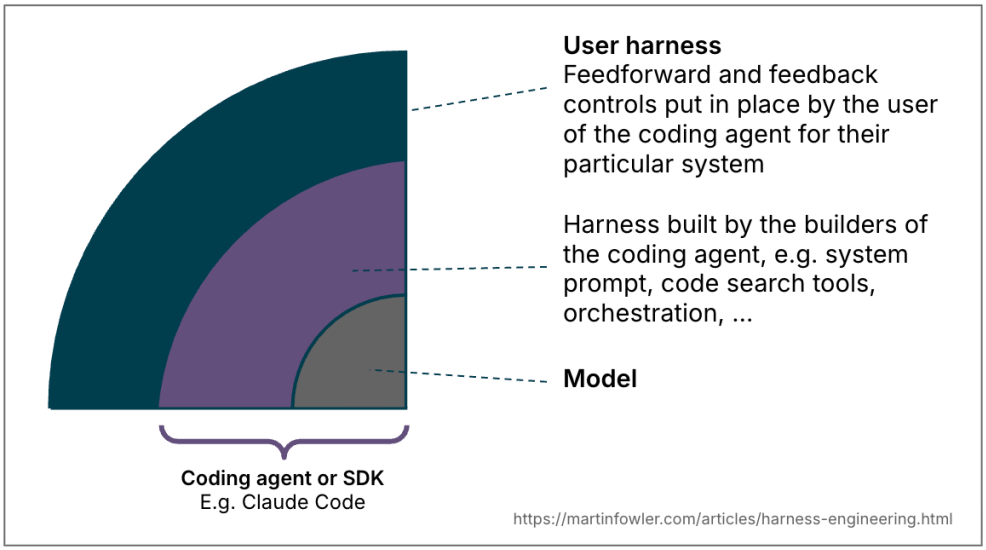
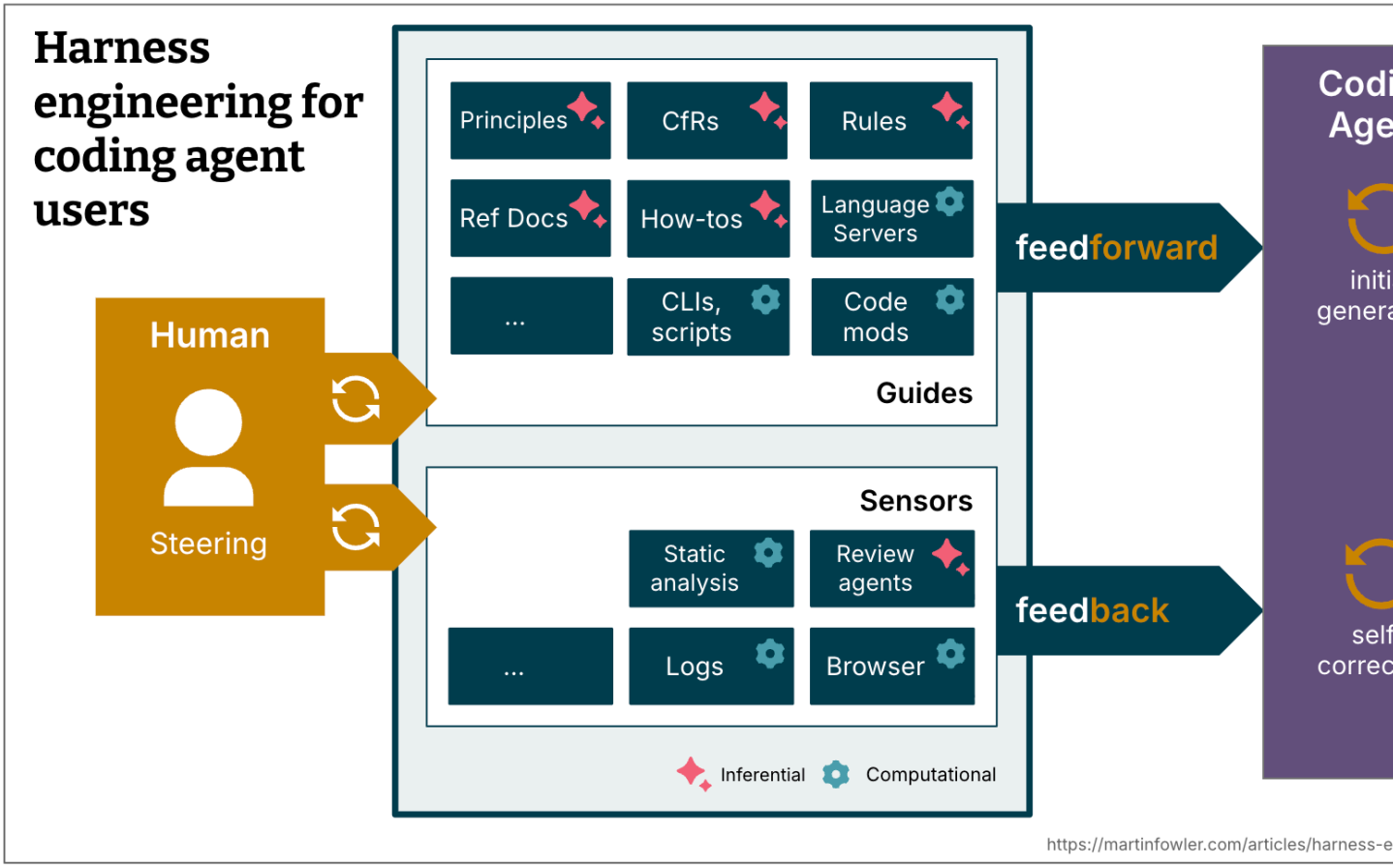


Figure 1: The term “harness” means different things depending on the bounded context.

A well-built outer harness serves two goals: it increases the probability that the agent gets it right in the first place, and it provides a feedback loop that self-corrects as many issues as possible before they even reach human eyes. Ultimately it should reduce the review toil and increase the system quality, all with the added benefit of fewer wasted tokens along the way.



Feedforward and Feedback

To harness a coding agent we both anticipate unwanted outputs and try to prevent them, and we put sensors in place to allow the agent to self-correct:

- **Guides (feedforward controls)** - anticipate the agent's behaviour and aim to steer it *before* it acts. Guides increase the probability that the agent creates good results in the first attempt
- **Sensors (feedback controls)** - observe *after* the agent acts and help it self-correct. Particularly powerful when they produce signals that are optimised for LLM consumption, e.g. custom linter messages that include instructions for the self-correction - a positive kind of prompt injection.

Separately, you get either an agent that keeps repeating the same mistakes (feedback-only) or an agent that encodes rules but never finds out whether they worked (feed-forward-only).

Computational vs Inferential

There are two execution types of guides and sensors:

- **Computational** - deterministic and fast, run by the CPU. Tests, linters, type checkers, structural analysis. Run in milliseconds to seconds; results are reliable.
- **Inferential** - Semantic analysis, AI code review, "LLM as judge". Typically run by a GPU or NPU. Slower and more expensive; results are more non-deterministic.

Computational guides increase the probability of good results with deterministic tooling. Computational sensors are cheap and fast enough to run on every change, alongside the agent. Inferential controls are of course more expensive and non-deterministic, but allow us to both provide rich guidance, and add additional semantic judgment. In spite of their non-determinism, inferential sensors can particularly increase our trust when used with a strong model, or rather a model that is suitable to the task at hand.

Examples

	Direction	Computational / Inferential	Example implementations
Coding conventions	feedforward	Inferential	AGENTS.md, Skills
Instructions how to bootstrap a new project	feedforward	Both	Skill with instructions and a bootstrap script
Code mods	feedforward	Computational	A tool with access to OpenRewrite recipes
Structural tests	feedback	Computational	A pre-commit (or coding agent) hook running

	Direction	Computational / Inferential	Example implementations
			ArchUnit tests that check for violations of module boundaries
Instructions how to review	feedback	Inferential	Skills

How does harness engineering relate to context engineering?

Context engineering provides us with the means to make guides and sensors available to the agent. Engineering a user harness for a coding agent is a specific form of context engineering.

The steering loop

The human's job in this is to **steer** the agent by iterating on the harness. Whenever an issue happens multiple times, the feedforward and feedback controls should be improved to make the issue less probable to occur in the future, or even prevent it.

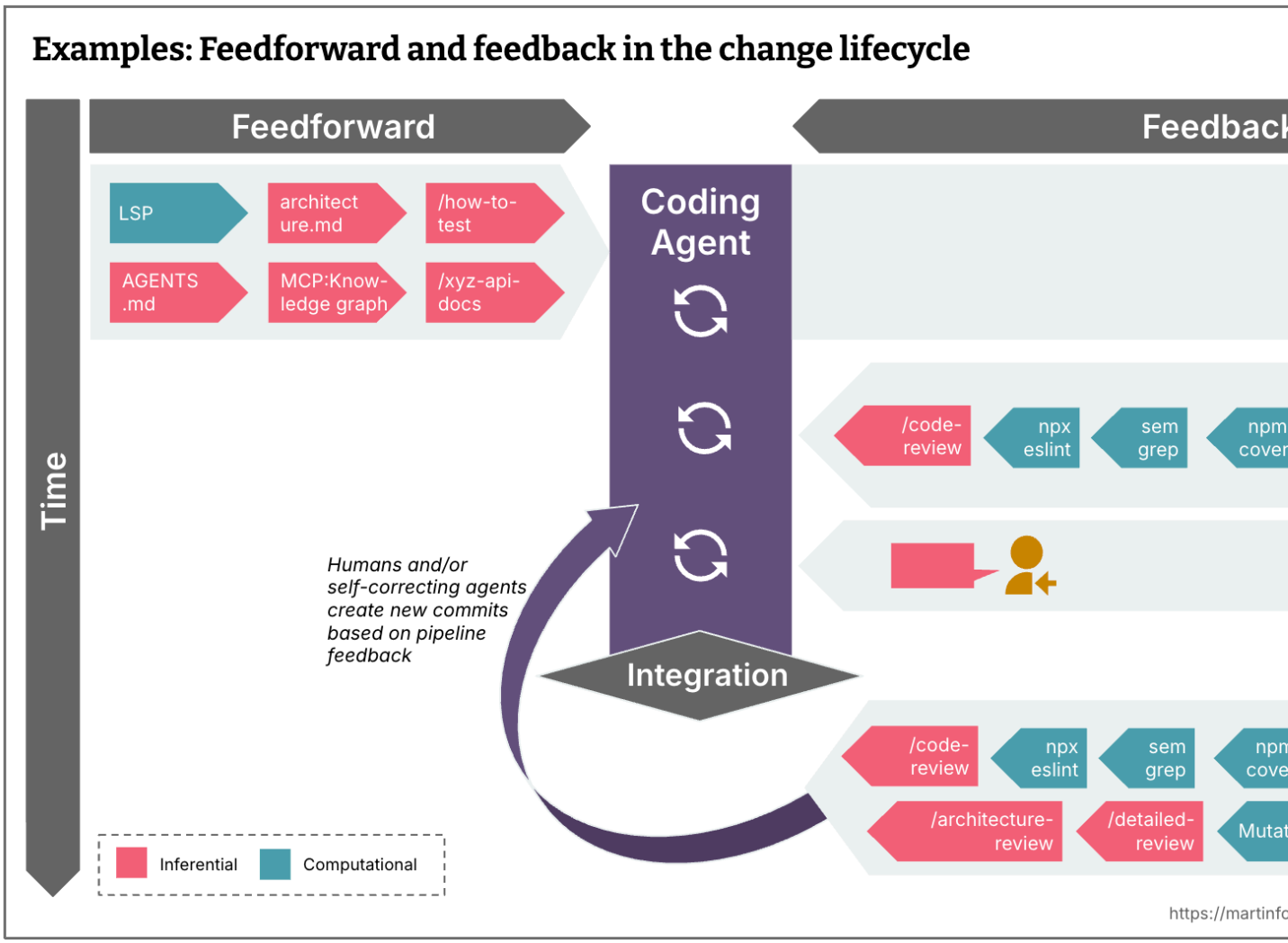
In the steering loop, we can of course also use AI to improve the harness. Coding agents now make it much cheaper to build more custom controls and more custom static analysis. Agents can help write structural tests, generate draft rules from observed patterns, scaffold custom linters, or create how-to guides from codebase archaeology.

Timing: Keep quality left

Teams who are continuously integrating have always faced the challenge of spreading tests, checks and human reviews across the development timeline according to their cost, speed and criticality. When you aspire to continuously deliver, you ideally even want every commit state to be deployable. You want to have checks as far left in the path to production as possible, since the earlier you find issues, the cheaper they are to fix. Feedback sensors, including the new inferential ones, need to be distributed across the lifecycle accordingly.

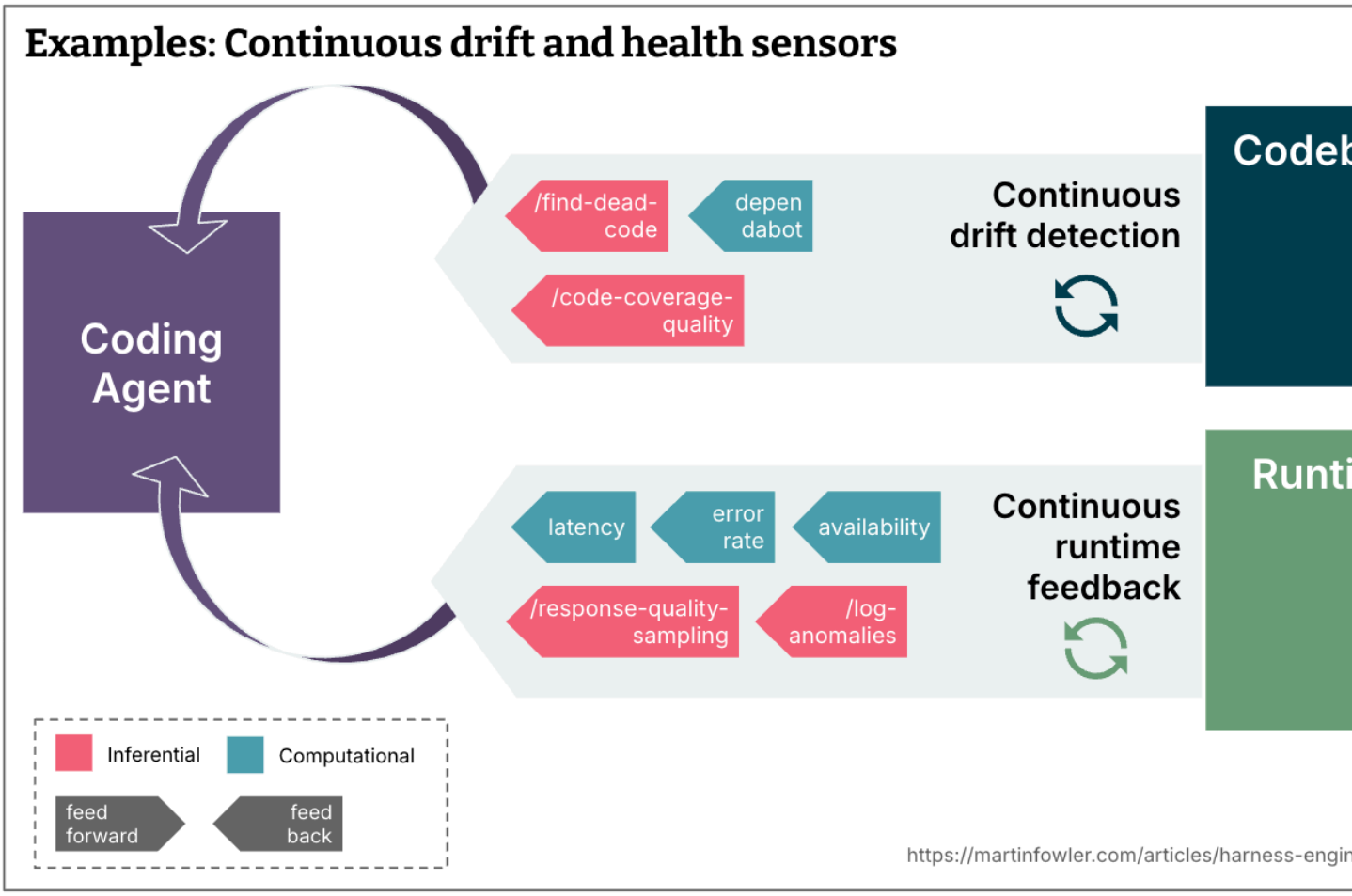
Feedforward and feedback in the change lifecycle

- What is reasonably fast and should be run even before integration, or even before a commit is even created? (e.g. linters, fast test suites, basic code review agent)
- What is more expensive and should therefore only be run post-integration in the pipeline, in addition to a repetition of the fast controls? (e.g. mutation testing, a more broad code review that can take into account the bigger picture)



Continuous drift and health sensors

- What type of drift accumulates gradually and should be monitored by sensors running continuously against the codebase, outside the change lifecycle? (e.g. dead code detection, analysis of the quality of the test coverage, dependency scanners)
- What runtime feedback could agents be monitoring? (e.g. having them look for degrading SLOs to make suggestions how to improve them, or AI judges continuously sampling response quality and flagging log anomalies)



Regulation categories

The agent harness acts like a cybernetic governor, combining feed-forward and feedback to regulate the codebase towards its desired state. It's useful to distinguish between multiple dimensions of that desired state, categorised by what the harness is supposed to regulate. Distinguishing between these categories helps because harnessability and complexity vary across them, and qualifying the word gives us more precise language for a term that is otherwise very generic.

The following are three categories that seem useful to me as of now:

Maintainability harness

More or less all of the examples I am giving in this article are about regulating internal code quality and maintainability. This is at the moment the easiest type of harness, as we have a lot of pre-existing tooling that we can use for this.

To reflect on how much these aforementioned maintainability harness ideas increase my trust in agents, I mapped common coding agent failure modes that I catalogued before against it.

Computational sensors catch the structural stuff reliably: duplicate code, cyclomatic complexity, missing test coverage, architectural drift, style violations. These are cheap, proven, and deterministic.

LLMs can partially address problems that require semantic judgment - semantically duplicate code, redundant tests, brute-force fixes, over-engineered solutions - but expensively and probabilistically. Not on every commit.

Neither catches reliably some of the higher-impact problems: Misdiagnosis of issues, overengineering and unnecessary features, misunderstood instructions. They'll sometimes catch them, but not reliably enough to reduce supervision. Correctness is outside any sensor's remit if the human didn't clearly specify what they wanted in the first place.

Architecture fitness harness

This groups guides and sensors that define and check the architecture characteristics of the application. Basically: Fitness Functions.

Examples:

- Skills that feed forward our performance requirements, and performance tests that feed back to the agent if it improved or degraded them.
- Skills that describe coding conventions for better observability (like logging standards), and debugging instructions that ask the agent to reflect on the quality of the logs it had available.

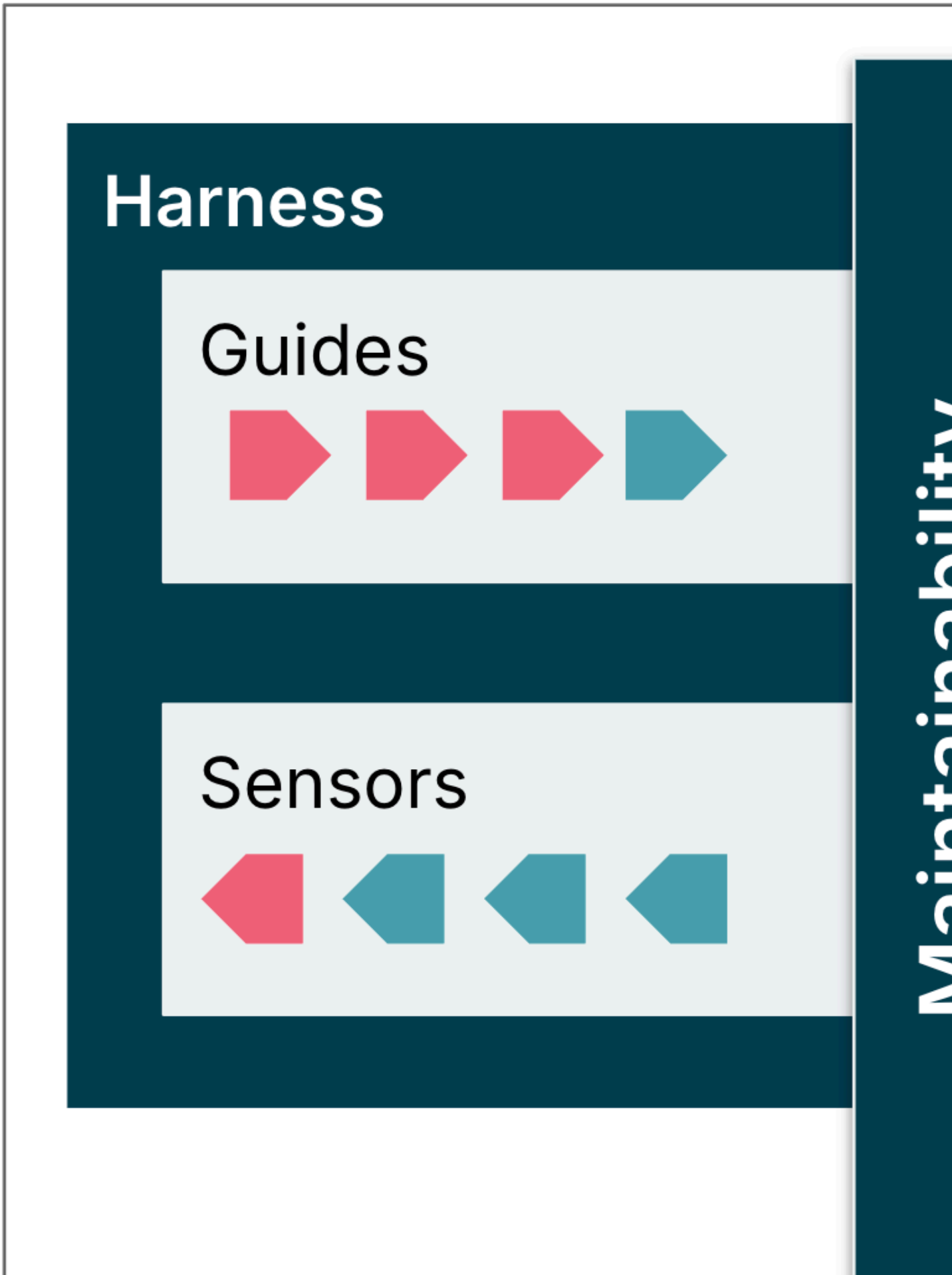
Behaviour harness

This is the elephant in the room - how do we guide and sense if the application functionally behaves the way we need it to? At the moment, I see most people who give high autonomy to their coding agents do this:

- Feed-forward: A functional specification (of varying levels of detail, from a short prompt to multi-file descriptions)
- Feed-back: Check if the AI-generated test suite is green, has reasonably high coverage, some might even monitor its quality with mutation testing. Then combine that with manual testing.

This approach puts a lot of faith into the AI-generated tests, that's not good enough yet. Some of my colleagues are seeing good results with the approved fixtures pattern, but it's easier to apply in some areas than others. They use it selectively where it fits, it's not a wholesale answer to the test quality problem.

So overall, we still have a lot to do to figure out good harnesses for functional behaviour that increase our confidence enough to reduce supervision and manual testing.


Harness
Guides
Sensors
Maintainability

Harnessability

Not every codebase is equally amenable to harnessing. A codebase written in a strongly typed language naturally has type-checking as a sensor; clearly definable module boundaries afford architectural constraint rules; frameworks like Spring abstract away details the agent doesn't even have to worry about and therefore implicitly increase the agent's chances of success. Without those properties, those controls aren't available to build.

Ambient affordances

My colleague Ned Letcher uses the term “ambient affordances” for the properties of the agent environment that make it more harnessable: “structural properties of the environment itself that make it legible, navigable, and tractable to agents operating within it.”

This plays out differently for greenfield versus legacy. Greenfield teams can bake harnessability in from day one - technology decisions and architecture choices determine how governable the codebase will be. Legacy teams, especially with applications that have accrued a lot of technical debt, face the harder problem: the harness is most needed where it is hardest to build.

Harness templates

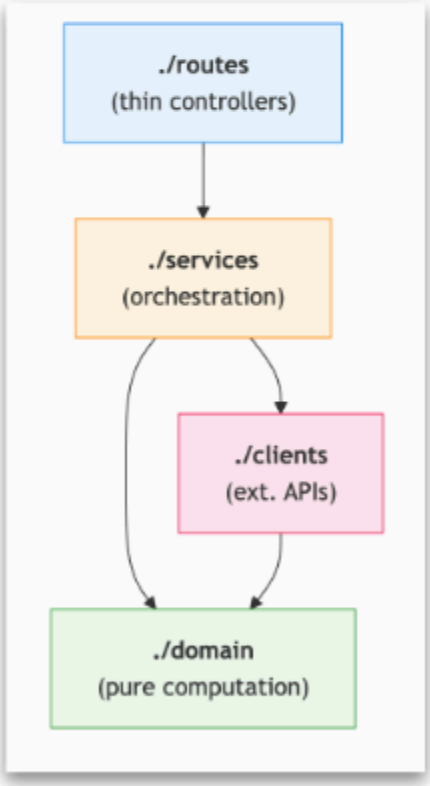
Most enterprises have a few common topologies of services that cover 80% of what they need - business services that exposes data via APIs; event processing services; data dashboards. In many mature engineering organizations these topologies are already codified in service templates. These might evolve into harness templates in the future: a bundle of guides and sensors that leash a coding agent to the structure, conventions and tech stack of a topology. Teams may start picking tech stacks and structures partly based on what harnesses are already available for them.

Event processor | Golang

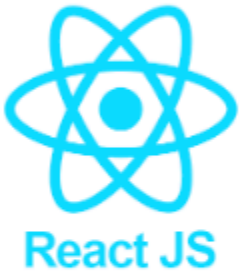
CRUD Business Service | JVM

Data dashboard | Node

Structure



Tech Sta



Ashby's Law

Ashby's Law of Requisite Variety is another interesting argument for these pre-defined topologies. The law says that a regulator must have at least as much variety as the system it governs, and it can only regulate what it has a model of. An LLM-based coding agent can produce almost anything, but committing to a topology narrows that space, making a comprehensive harness more achievable. Defining topologies is a variety-reduction move.

We would of course face similar challenges as with service templates. As soon as teams instantiate them, they start fall out of sync with upstream improvements. Harness templates would face the same versioning and contribution problems, maybe even worse with non-deterministic guides and sensors that are harder to test.

The role of the human

As human developers we bring our skills and experience as an implicit harness to every codebase. We absorbed conventions and good practices, we have felt the cognitive pain of complexity, and we know that our name is on the commit. We also carry organisational alignment - awareness of what the team is trying to achieve, which technical debt is tolerated for business reasons, and what “good” looks like in this specific context. We go in small steps and at our human pace, which creates the thinking space for that experience to get triggered and applied.

A coding agent has none of this: no social accountability, no aesthetic disgust at a 300-line function, no intuition that “we don't do it that way here,” and no organisational memory. It doesn't know which convention is load-bearing and which is just habit, or whether the technically correct solution fits what the team is trying to do.

Harnesses are an attempt to externalise and make explicit what human developer experience brings to the table, but it can only go so far. Building a coherent system of guides and sensors and self-correction loops is expensive, so we have to prioritise with a clear goal in mind: A good harness should not necessarily aim to fully eliminate human input, but to direct it to where our input is most important.

A starting point - and open questions

The mental model I've laid out here describes techniques that are already happening in practice and helps frame discussions about what we still need to figure out. Its goal is to raise the conversation above the feature level - from skills and MCP servers to how we strategically design a system of controls that gives us genuine confidence in what agents produce.

Here are some harness-related examples from the current discourse:

- An OpenAI team documented what their harness looks like: layered architecture enforced by custom linters and structural tests, and recurring "garbage collection" that scans for drift and has agents suggest fixes. Their conclusion: "Our most difficult challenges now center on designing environments, feedback loops, and control systems."
- Stripe's write-up about their minions describes things like pre-push hooks that run relevant linters based on a heuristic, they highlight how important "shift feedback left" is to them, and their "blueprints" show how they're integrating feedback sensors into the agent workflows.
- Mutation and structural testing are examples of computational feedback sensors that have been underused in the past, but are now having a resurgence.
- There is increased chatter among developers about the integration of LSPs and code intelligence in coding agents, examples of computational feedforward guides.
- I hear stories from teams at Thoughtworks about tackling architecture drift with both computational and inferential sensors, e.g. increasing API quality with a mix of agents and custom linters, or increasing code quality with a "janitor army".

There's plenty still to figure out, not just the already mentioned behavioural harness. How do we keep a harness coherent as it grows, with guides and sensors in sync, not contradicting each other? How far can we trust agents to make sensible trade-offs when instructions and feedback signals point in different directions? If sensors never fire, is that a sign of high quality or inadequate detection mechanisms? We need a way to evaluate harness coverage and quality similar to what code coverage and mutation testing do for tests. Feedforward and feedback controls are currently scattered across delivery steps, there's real potential for tooling that helps configure, sync, and reason about them as a system. Building this outer harness is emerging as an ongoing engineering practice, not a one-time configuration.

Acknowledgements

Big thanks to the Doppler team for the engaging discussion at our last technology radar meeting, in particular Kief Morris for bringing up cybernetics. Thanks to Ned Letcher, Chris Ford and Ben O'Mahoney for the conversations about what a harness even is, and to Matteo Vaccari for his insights on the behaviour harness. And to everybody who took the time to read the draft and provide lots of valuable feedback: Christoph Burgmer, Jörn Dinkla, Michael Feathers, Karrtik Iyer, Swapnil Phulse, Paul Sobocinski, Zhenjia Zhou

GenAI (Claude and Claude Code) was used for research, pulling in relevant ideas from existing notes, and polishing the language.

Earlier Memo

I wrote a memo in early February containing my initial thoughts on Harness Engineering as the term first appeared. That post has attracted a lot of traffic. This article supersedes that memo, so we have redirected the original memo URL to this page, as we believe this page is the better resource for readers.

Significant Revisions

02 April 2026: published full article including introducing guides, sensors, computational and inferential elements, and harness templates

17 February 2026: published my initial memo on Harness Engineering

© Martin Fowler | Disclosures